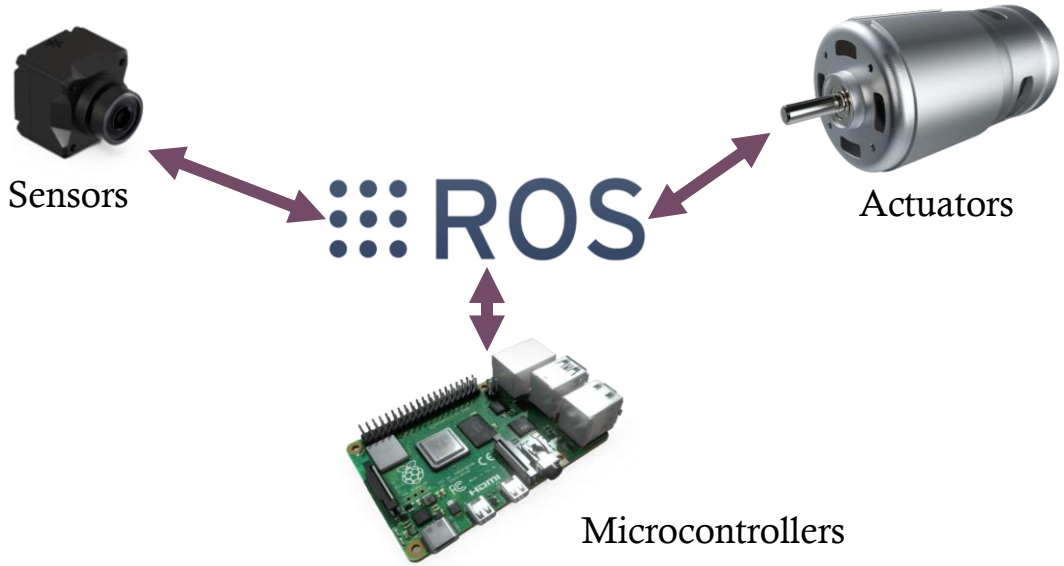
M2 T251F052 PHAN PHEARAMONY
Kamal's Laboratory
Gunma University

ROS2 ROBOTICS WORKSHOP

WHAT IS ROS2?

ROS : Robotics Operating System

ROS - an ecosystem that provides the framework, tools, and libraries, for building, deploying, running, and maintaining robotic applications.



Example:

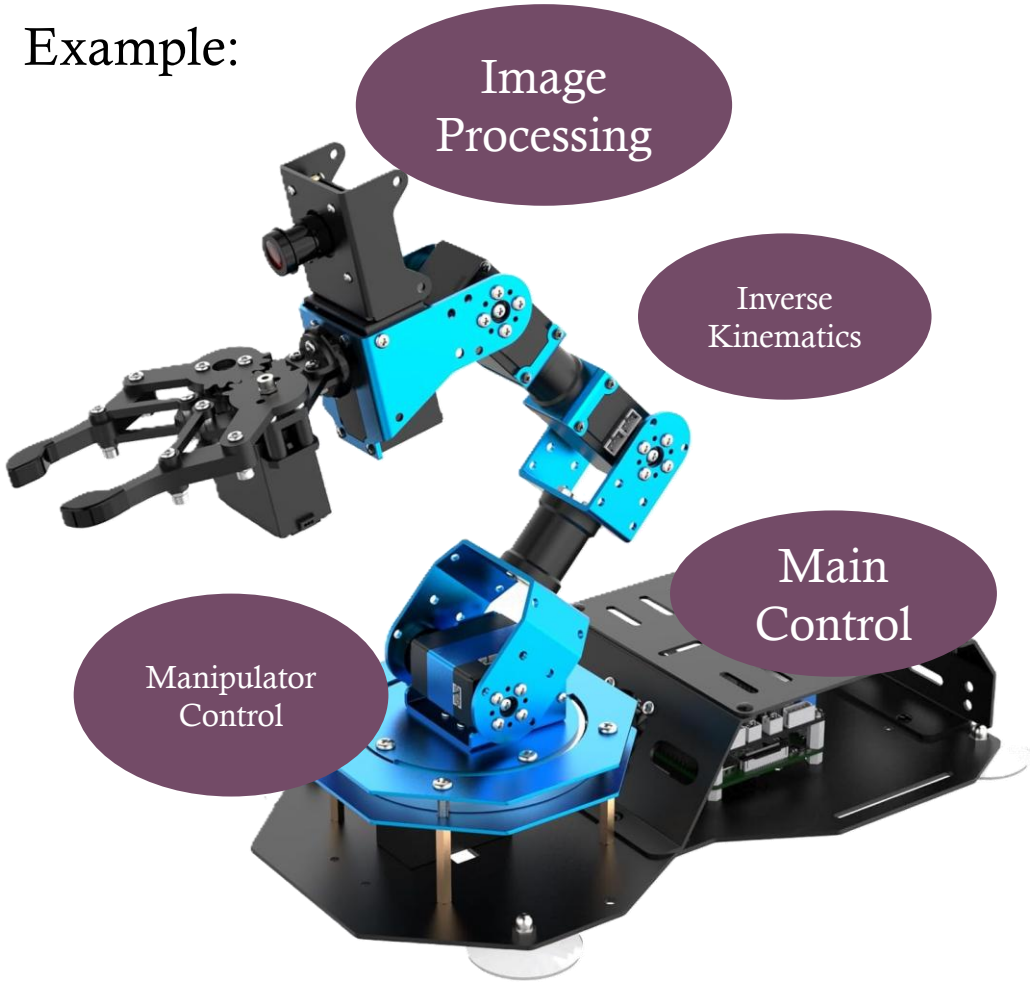
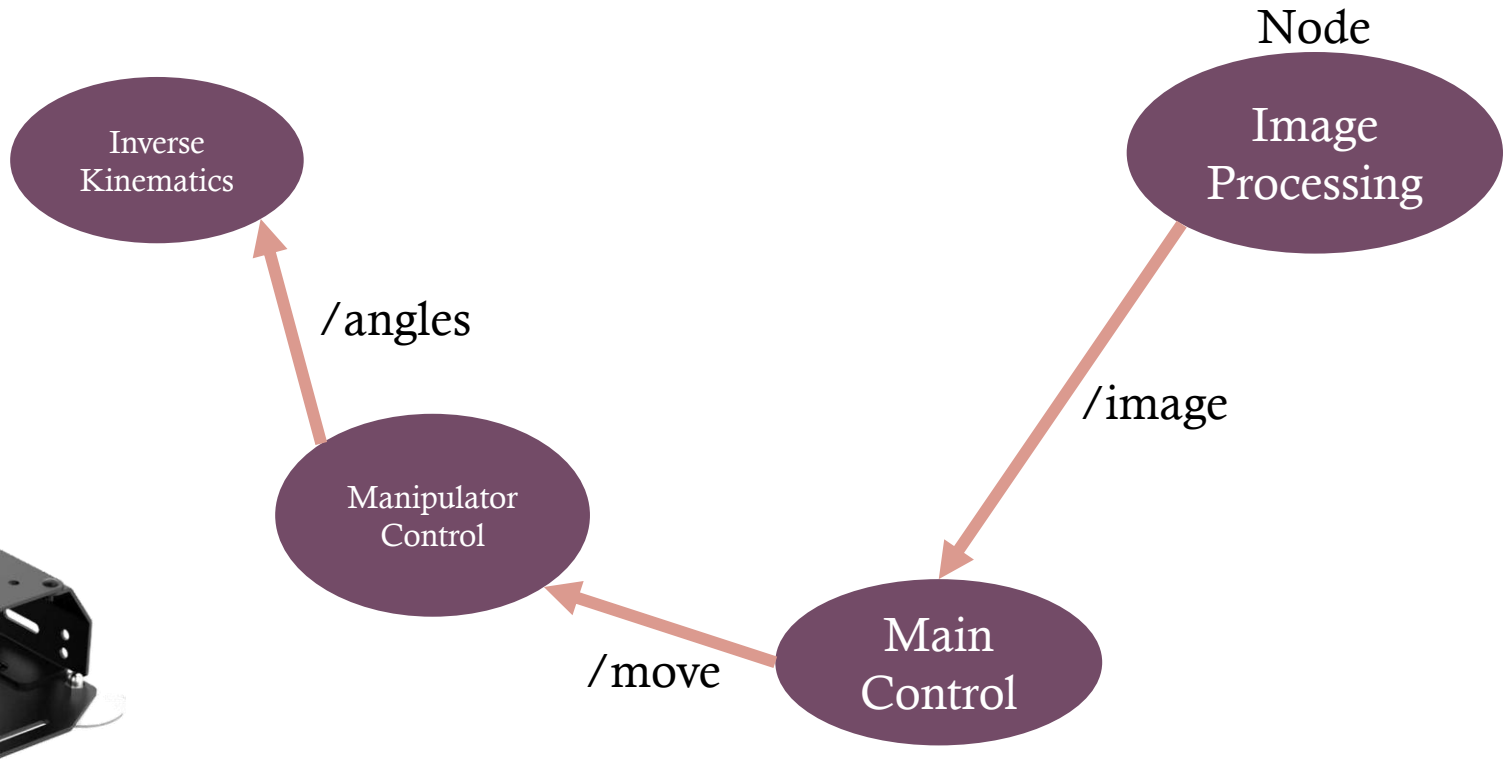
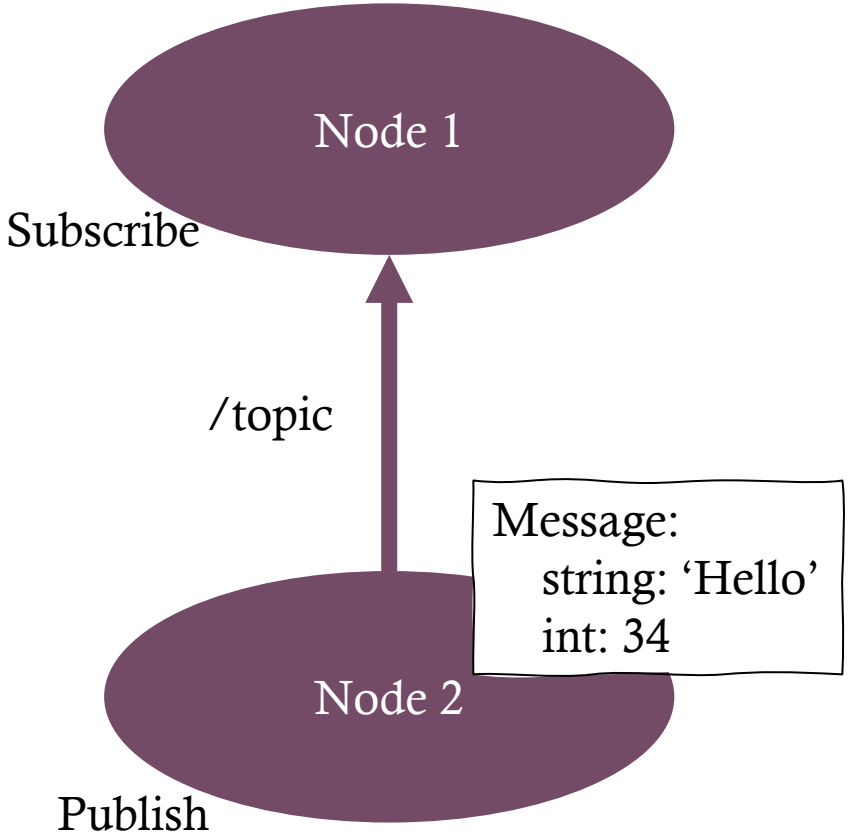


Image Processing
Inverse Kinematics
Main Control
Manipulator Control

Example:

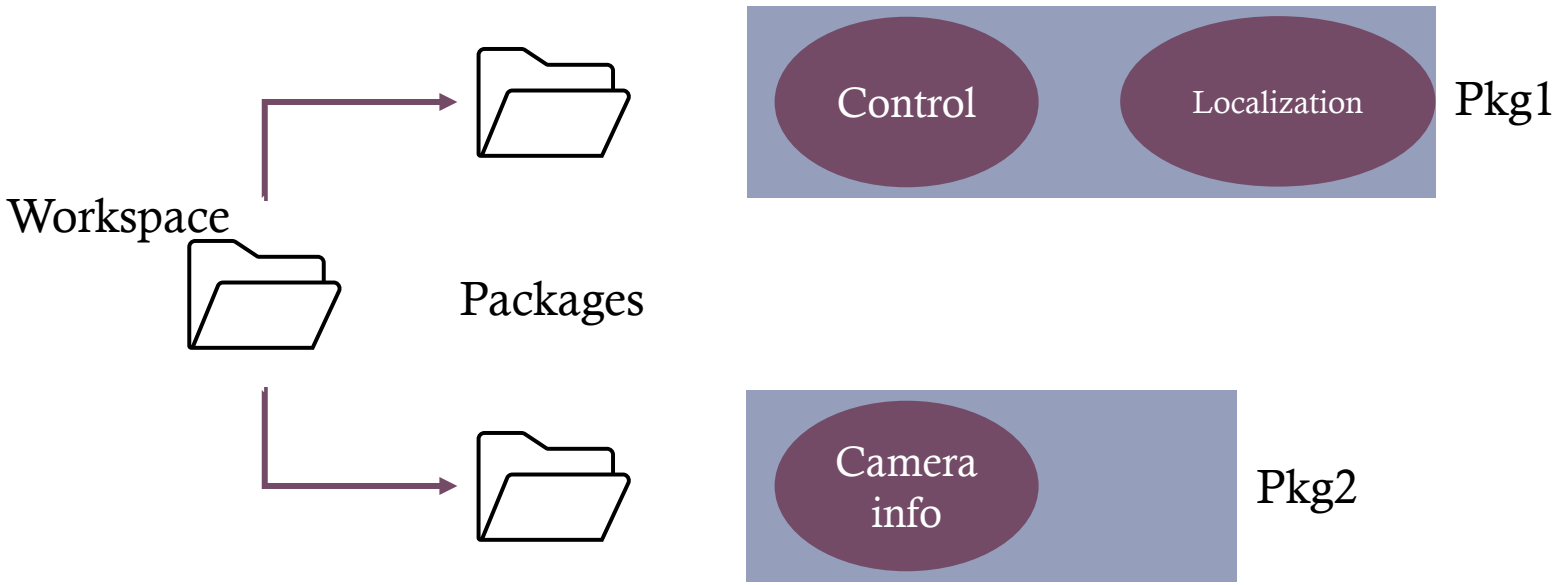

Node
Image Processing
Inverse Kinematics
/angles
/image
Manipulator Control
/move
Main Control

HOW NODES COMMUNICATES?

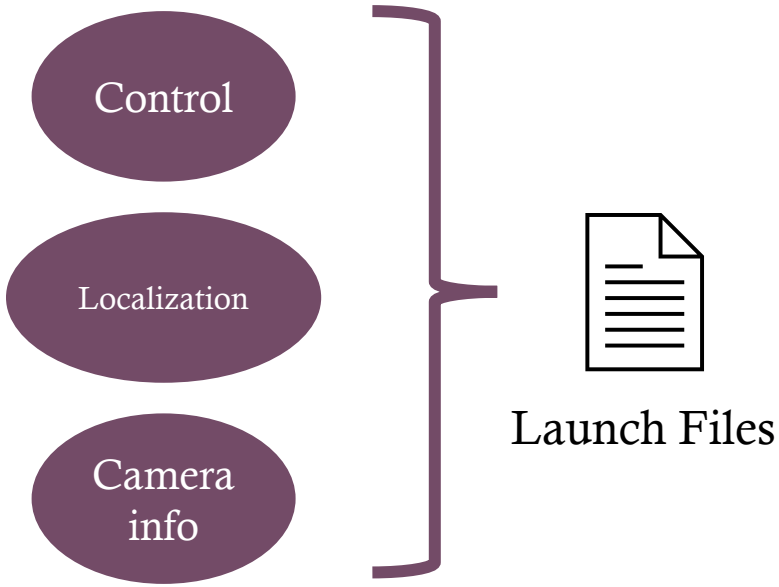


Topic	Message Type	Description	Dir
/scan	sensor_msgs/LaserScan	LiDAR obstacle data	PUB
/odom	nav_msgs/Odometry	Robot position estimate	PUB
/imu	sensor_msgs/Imu	Angular velocity (yaw rate)	PUB
/cmd_vel	geometry_msgs/Twist	Velocity command to robot	SUB
/map	nav_msgs/OccupancyGrid	2D occupancy grid map	PUB
/planned_path	nav_msgs/Path	A* computed path	PUB
/camera/image_raw	sensor_msgs/Image	Raw camera feed	PUB

ROS2 STRUCTURE



ROS2 STRUCTURE



Built using “Colcon build”

GETTING STARTED: SETTING UP



ROS Humble (2022), Jazzy (2024), Kilted Kaiju (2025)

GETTING STARTED: SETTING UP

Ubuntu 22.04

+

Humble (2022)

Running on Windows

GETTING STARTED: SETTING UP

Windows

Install WSL2 (Running Ubuntu on Windows):

wsl –install

→Restart your PC

Install Ubuntu 22.04

wsl –list –online

wsl --install -d Ubuntu-22.04

Opening WSL

wsl

GETTING STARTED: SETTING UP

Ubuntu 22.04

Install ROS 2 Humble

sudo apt update
sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8
LANG=en_US.UTF-8
export LANG=en_US.UTF-8

---

sudo apt install software-properties-common
sudo add-apt-repository universe

---

sudo apt update
sudo apt install curl gnupg lsb-release -y

GETTING STARTED: SETTING UP

Ubuntu 22.04

```
sudo curl -sSL
https://raw.githubusercontent.com/ros/rosdistro/master/ros.ke
y \
-o /usr/share/keyrings/ros-archive-keyring.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-
archive-keyring.gpg] \
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo
$UBUNTU_CODENAME) main" | \
sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

```
sudo apt update
sudo apt install ros-humble-desktop
```

GETTING STARTED: SETTING UP

Ubuntu 22.04

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

```
sudo apt install python3-colcon-common-extensions python3-rosdep
```

```
sudo rosdep init
rosdep update
```

Test ROS 2

On Terminal 1:

```
ros2 run demo_nodes_cpp talker
```

On Terminal 2:

```
ros2 run demo_nodes_py listener
```

GETTING STARTED: SETTING UP

Browsers

http://10.21.89.212:6081/vnc.html

Password: student

http://10.21.89.212:6095/vnc.html

VNC

SSH:

ssh student@10.21.89.212 –p 2202

ssh student@10.21.89.212 –p 2215

SETTING UP WORKSPACE

Structures:

ros2_ws/

src/ : packages go here

build/ : auto generated by colcon

install/ : auto generated by colcon

log/ : auto generated by colcon

Source ROS2: tell terminal where ROS2 is installed

```
source /opt/ros/humble/setup.bash
```

Create workspace folder

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
```

Build it

```
colcon build
```

Source the workspace

```
source ~/ros2_ws/install/setup.bash
```

CREATE PACKAGE

Create the first package

```
cd ~/ros2_ws/src
ros2 pkg create my_robot \
--build-type ament_python \
--dependencies rclpy std_msgs geometry_msgs
```

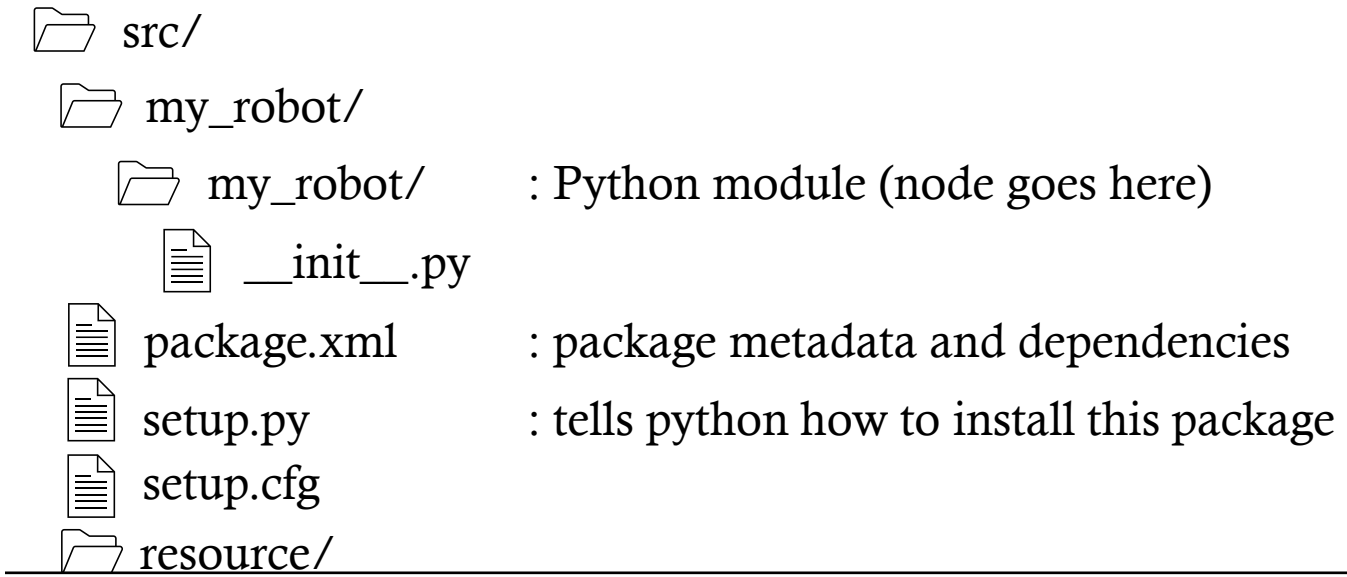
← Move to src

← Create a package called my_robot

← Tell ROS2 we are writing in Python

← Tell ROS2 we are using these libraries

Structures:



BUILD THE FIRST NODE: HELLO ROBOT

nano ~/ros2_ws/src/my_robot/my_robot/hello_robot.py

```
import rclpy
from rclpy.node import Node

class HelloRobot(Node):          # all nodes inherit from Node

    def __init__(self):
        super().__init__('hello_robot')   # node name

        self.get_logger().info('Hello from ROS2!')
        # create a timer: call self.tick every 1.0 seconds
        self.create_timer(1.0, self.tick)

    def tick(self):
        self.get_logger().info('Robot is alive!')

def main(args=None):
    rclpy.init(args=args)          # start ROS2
    node = HelloRobot()            # create our node
    rclpy.spin(node)               # keep it running
    node.destroy_node()            # clean up
    rclpy.shutdown()               # stop ROS2
```

BUILD THE FIRST NODE: HELLO ROBOT

Register the Node:

nano ~/ros2_ws/src/my_robot/setup.py

```
entry_points={
    'console_scripts': [
        'hello_robot = my_robot.hello_robot:main',
        #  ^^^name      ^^^module.file  ^^^function
    ],
},
```

Build and Run:

```
cd ~/ros2 ws && colcon build
source install/setup.bash
ros2 run my_robot hello_robot
```

Expected output:

```
[INFO] [hello_robot]: Hello from ROS2!
[INFO] [hello_robot]: Robot is alive!
[INFO] [hello_robot]: Robot is alive!
...
```

PUBLISHER & SUBSCRIBER

Create Publisher Node:

nano ~/ros2_ws/src/my_robot/my_robot/counter_publisher.py

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Int32     # message type: a single integer

class CounterPublisher(Node):

    def __init__(self):
        super().__init__('counter_publisher')

        # Create a publisher on topic '/counter', message type Int32
        self.pub = self.create_publisher(Int32, '/counter', 10)
        #                                ^^^^    ^^^^^^^     ^^ queue size

        self.count = 0
        self.create_timer(1.0, self.publish_count)
```

```
    def publish_count(self):
        msg = Int32()              # create an empty Int32 message
        msg.data = self.count      # fill in the value
        self.pub.publish(msg)      # send it
        self.get_logger().info(f'Publishing: {self.count}')
        self.count += 1

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(CounterPublisher())
    rclpy.shutdown()
```

PUBLISHER & SUBSCRIBER

Create Subscriber Node:

nano ~/ros2_ws/src/my_robot/my_robot/counter_subscriber.py

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Int32

class CounterSubscriber(Node):

    def __init__(self):
        super().__init__('counter_subscriber')

        # Subscribe to '/counter', call self.on_msg when data arrives
        self.sub = self.create_subscription(
            Int32, '/counter', self.on_msg, 10)

    def on_msg(self, msg):
        self.get_logger().info(f'Received: {msg.data}')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(CounterSubscriber())
    rclpy.shutdown()
```

Register Both Nodes:

```
'console_scripts': [
    'hello_robot       = my_robot.hello_robot:main',
    'counter_publisher  = my_robot.counter_publisher:main',
    'counter_subscriber = my_robot.counter_subscriber:main',
],
```

PUBLISHER & SUBSCRIBER

Build and Run:

cd ~/ros2_ws && colcon build && source install/setup.bash

Terminal 1:

ros2 run my_robot counter_publisher

Terminal 2:

ros2 run my_robot counter_subscriber

Expected output:

```
# Terminal 1:
[INFO] [counter_publisher]: Publishing: 0
[INFO] [counter_publisher]: Publishing: 1
```

```
# Terminal 2:
[INFO] [counter_subscriber]: Received: 0
[INFO] [counter_subscriber]: Received: 1
```

PUBLISHER & SUBSCRIBER

Check topic:

ros2 topic list

Expected output:

```
/counter
/parameter_events
/rosout
```

Echo topic (listening to topic):

ros2 topic echo /counter

Expected output:

```
data: 98
---
data: 99
---
data: 100
---
data: 101
---
```

CREATE LAUNCH FILES: STARTING MULTIPLE NODES AT ONCE

Create launch folder:

mkdir -p ~/ros2_ws/src/my_robot/launch

Create launch file:

nano ~/ros2_ws/src/my_robot/launch/talker_listener.launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([

        Node(
            package='my_robot',
            executable='counter_publisher',
            name='counter_publisher',
            output='screen',
        ),

        Node(
            package='my_robot',
            executable='counter_subscriber',
            name='counter_subscriber',
            output='screen',
        ),

    ])
```

CREATE LAUNCH FILES: STARTING MULTIPLE NODES AT ONCE

Tell setup.py where the launch folder is:

nano ~/ros2_ws/src/my_robot/setup.py

```
import os
from glob import glob
from setuptools import find_packages, setup

setup(
    ...
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        # Add this line:
        (os.path.join('share', package_name, 'launch'),
            glob('launch/*.launch.py')),
    ],
    ...
)
```

CREATE LAUNCH FILES: STARTING MULTIPLE NODES AT ONCE

Build and Launch

cd ~/ros2_ws && colcon build && source install/setup.bash

ros2 launch my_robot talker_listener.launch.py

Expected output:

```
[counter_publisher]: Publishing: 0
[counter_subscriber]: Received: 0
[counter_publisher]: Publishing: 1
[counter_subscriber]: Received: 1
```